

SmartCare v0.5 - Architecture and Refactoring Workbook

Week 8 student resource

Current Architecture Problems

Problem	Evidence	Impact	Refactoring
Lack of an overarching application orchestrator (ClinicController)	Creating a booking currently requires manual coordination, creating dictionaries, and passing raw list references across individual Patient, Practitioner, and Appointment instances in test scripts.	High coupling between testing/UI harnesses and individual domain class mechanics; leaks orchestration logic into client interfaces and leads to code duplication across different entry points.	Implement ClinicController as an aggregate root to manage _patients, _practitioners, and _appointments registries and expose unified entry-point methods like book_appointment().
Weak guardrails during direct instantiation (init) of Appointment objects regarding status invariants.	init allows instantiating an Appointment directly with status = AppointmentStatus.CANCELLED while leaving cancellation_reason = ""	Corrupts domain model state integrity; allows invalid or incomplete domain objects to exist in system memory.	Add an explicit validation guard in __init__:
Risk of embedding persistence operations (like SQL execution) or I/O methods (input(), print()) directly inside domain entities or controller methods as storage requirements are added.	The current domain models (Appointment, Patient, Practitioner) are clean domain objects, but planned SQLite persistence could tempt placing sqlite3 execution inside entity methods if layer boundaries aren't explicitly enforced.	Violates the Single Responsibility Principle (SRP) and Dependency Inversion Principle (DIP); tightly couples domain business logic to database infrastructure, making unit testing without a live database impossible.	Introduce a dedicated Persistence / Repository Layer (AppointmentRepository) using constructor dependency injection to handle SQLite queries completely outside the domain classes.
Over-reliance on strict, un-normalized string equality checks within collision logic.	Appointment.check_collision() evaluates appointment.practitioner_id == practitioner_id and appointment.date_time == date_time using raw string comparison.	False negatives in collision detection. Incidental trailing spaces or casing discrepancies (e.g., "P001 " vs "p001" or "2026-10-15 09:00 " vs "2026-10-15 09:00") will bypass collision checks and allow	Apply string normalization inside check_collision():

		double-booking a doctor.	
--	--	--------------------------	--

Layer Responsibilities

Layer	Responsibilities	Must not contain
Presentation	• Captures user inputs (e.g., input(), menu clicks, command-line arguments). • Formats and displays output messages to the user (e.g., print(), tables, dialogs). • Converts raw user text strings into application calls.	• Direct business rules, state transition validation, or collision checks. • Direct database queries (SQL statements) or connection strings. • Direct mutation of private entity attributes.
Service	• Orchestrates high-level use cases (e.g., book_appointment, register_patient). • Maintains central registries and verifies foreign entity existence before operations. • Coordinates interactions between Domain entities and Repositories via Dependency Injection.	• Terminal input() or print() calls • Direct SQL queries or database driver execution (sqlite3). • Duplicate state machine logic or core entity validation rules that belong inside domain objects.
Domain	• Represents core business concepts (Patient, Practitioner, Appointment). • Enforces internal state invariants, lifecycle rules (ALLOWED_TRANSITIONS), and validation. • Contains pure, deterministic business logic (e.g., check_collision(), cancel_appointment()).	• SQL code, database connections, or ORM dependencies. • UI presentation logic, string formatting for display, or terminal I/O. • External framework annotations or network call dependencies.
Repository	• Exposes clean object-oriented data retrieval and storage interfaces (e.g., find_by_id(), save()). • Translates database query results into domain entity objects. • Hides low-level persistence mechanisms behind standard collection-like methods.	• Business rules, state machine transitions, or collision checks. • User interface prompts or display formatting. • Direct application workflow orchestration logic.
Persistence	• Executes low-level SQL statements (INSERT, UPDATE, SELECT) against SQLite/RDBMS. • Manages database connections, transactions, database files, and table schemas. • Handles physical data storage and serialization.	• Application business rules, domain invariants, or state transition logic. • User-facing formatting or CLI prompts. • Direct references to UI layers.

Architecture Diagram

Insert SmartCare v0.5 architecture and dependency direction.

SEE IMAGE FILE IN THE SAME FOLDER FOR THE DIAGRAM

SOLID Review

Principle	Relevant?	Evidence	Decision
SRP	YES	Appointment focuses exclusively on domain logic (state machine transitions, collision validation, and cancellation reason rules). Persistence execution (SQL queries) and user interaction (input()/print()) are kept out of entity classes.	Adhere Strictly: Maintain separate layers so Appointment handles business rules, ClinicController handles application orchestration, AppointmentRepository manages SQL/database logic, and presentation views handle I/O.
OCP	YES	Adding new lifecycle states or transition paths is driven by data configuration (ALLOWED_TRANSITIONS dictionary) rather than sprawling if/else or switch statements across client code.	Adhere: Use state dictionaries or strategy patterns for extensible logic without modifying existing method structures when business rules expand.
LSP	YES	Refused making Appointment inherit from Patient or Practitioner; hypothetical specialized roles (e.g., Doctor extending Practitioner) maintain exact interface contracts without breaking base class assumptions.	Adhere: Avoid forced inheritance between distinct entities. Favor composition/associations for entities and reserve inheritance strictly for true "IS-A" relationships that satisfy full substitutability.
ISP	YES	Avoiding monolithic "fat" interfaces or forcing classes to depend on unused methods (e.g., avoiding a single 20-method repository when a component only requires read access).	Adhere Pragmatically: Keep component interfaces focused and lean. Do not force interfaces on domain entities where concrete implementations suffice, but segregate infrastructure capabilities as services grow.
DIP	YES	High-level application components (ClinicController) depend on repository abstractions rather than	Adhere: Use constructor dependency injection to pass persistence implementations into the application controller,

		importing low-level database drivers (sqlite3) directly inside domain classes.	enabling fast in-memory unit testing without database dependencies.
--	--	--	---

AI Architecture Review

AI suggestion	Observed problem?	Decision	Reason	Verification
Implement an Aggregate Root (ClinicController)	High-level orchestration, entity lookups, and foreign key validations were being handled inside client test scripts (manual_tester.py), causing code duplication and high coupling.	Accepted	Centralizes system use cases (book_appointment, register_patient) and maintains collection registries without scattering orchestration across client scripts.	Verified by checking that ClinicController delegates business rules to domain objects while handling registry key existence checks prior to booking instantiation.